

Iterators & Generators — JavaScript Sequence & Stream Architecture

JavaScript-এ ডেটা ক্রমান্বয়ে অ্যাক্সেস করার পেছনে একটি শক্তিশালী অন্তর্নিহিত মেকানিজম কাজ করে—যার নাম **Iteration Protocols**। এই চ্যাপ্টারে আমরা **Iterable**, **Iterator**, **Symbol.iterator**, **Generator Functions (function*)**, **yield**, **Lazy Evaluation** এবং ইনফিনিট স্ট্রিম প্রসেসিংয়ের গভীরতম খুঁটিনাটি একদম বাস্তব উদাহরণসহ নিখুঁতভাবে শিখব।

- | | | |
|-----------------------------|-------------------------------|-------------------------------|
| ★ 1. Iterable Concept | ★ 2. Iterator Protocol | ★ 3. Symbol.iterator |
| ★ 4. next() Mechanics | ★ 5. { value, done } | ★ 6. for...of Loop Internal |
| ★ 7. Custom Iterators | ★ 8. Generator Function* | ★ 9. yield Operator |
| ★ 10. Pause & Resume | ★ 11. Generator State Machine | ★ 12. Two-way next(val) |
| ★ 13. return() & throw() | ★ 14. yield* Delegation | ★ 15. Infinite Streams |
| ★ 16. Lazy Evaluation | ★ 17. Fibonacci Generator | ★ 18. Range Generator |
| ★ 19. Custom Iterable Class | ★ 20. Object as Iterable | ★ 21. API Pagination Stream |
| ★ 22. Async Generators | ★ 23. Memory Optimization | ★ 24. 9 Production Challenges |

Part 01 — Iterables, Iteration Protocols & Symbol.iterator (27.1 – 27.8)

27.1 প্রথমে একটা সমস্যা বুঝি

ধরো আমাদের কাছে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30, 40];
```

আমরা লিখতে পারি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for (const number of numbers) {  
  console.log(number);  
}
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
10  
20  
30  
40
```

কিন্তু প্রশ্ন হলো: > JavaScript কীভাবে জানে যে array-এর কোন value আগে দিতে হবে এবং কোনটা পরে দিতে হবে? এর পেছনে আছে **Iterator**!

27.2 Iterable কী?

Iterable হলো এমন কোনো value যেটাকে একটার পর একটা করে iterate করা যায়। Common iterable:

► CONSOLE OUTPUT

OUTPUT

```
Array  
String  
Map  
Set  
TypedArray  
Generator
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30];  
  
for (const number of numbers) {  
  console.log(number);  
}
```

Array হলো iterable।

27.3 String-ও Iterable

JAVASCRIPT SOURCE

JAVASCRIPT

```
const word = "HELLO";  
  
for (const letter of word) {  
  console.log(letter);  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
H  
E  
L  
L  
O
```

কারণ String-ও iterable।

27.4 Symbol.iterator

Iterable object-এর একটি বিশেষ method থাকে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Symbol.iterator
```

Array-এর ক্ষেত্রে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30];  
  
console.log(numbers[Symbol.iterator]);
```

Output function-এর মতো কিছু দেখতে পারো:

► CONSOLE OUTPUT

OUTPUT

```
[Function: values]
```

এই method একটি **iterator** return করে।

27.5 Iterator কী?

Iterator হলো এমন একটি object যেটা sequentially values provide করে। একটি iterator-এর মূল method:

JAVASCRIPT SOURCE

JAVASCRIPT

```
next()
```

Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20, 30];

const iterator = numbers[Symbol.iterator]();

console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
{ value: 10, done: false }
{ value: 20, done: false }
{ value: 30, done: false }
{ value: undefined, done: true }
```

🔴 next() কী return করে?

প্রতিবার next() একটি object return করে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
{
  value: ... ,
  done: ...
}
```

দুটি property:

Property	অর্থ
value	বর্তমান value
done	iteration শেষ হয়েছে কিনা

27.6 done বুঝি

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [10, 20];

const iterator = numbers[Symbol.iterator]();

console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
{ value: 10, done: false }
{ value: 20, done: false }
{ value: undefined, done: true }
```

শেষে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
done: true
```

মানে: > আর কোনো value নেই।

27.7 Iterator Manually চালানো

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [100, 200, 300];

const iterator = numbers[Symbol.iterator]();

let result = iterator.next();

while (!result.done) {

  console.log(result.value);

  result = iterator.next();

}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
100
200
300
```

অর্থাৎ `for ... of` মূলত এই ধরনের iteration mechanism ব্যবহার করে।

27.8 for ... of কীভাবে কাজ করে?

তুমি লিখলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for (const number of numbers) {
  console.log(number);
}
```

Conceptually:

🕒 ARCHITECTURE / CONCEPT FLOW

```
numbers
↓
Symbol.iterator
↓
iterator
↓
next()
↓
value
↓
next()
↓
value
↓
done === true
```

অর্থাৎ:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for ... of
```

এর পেছনে iterator protocol কাজ করছে।

27.9 Iterable বনাম Iterator

এটা খুব গুরুত্বপূর্ণ।

◆ Iterable

যার কাছে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
Symbol.iterator
```

থাকে।

◆ Iterator

যার কাছে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
next()
```

থাকে। সহজভাবে:

🕒 ARCHITECTURE / CONCEPT FLOW

Iterable → Iterator তৈরি করতে পারে

Iterator → next() দিয়ে value দেয়

27.10 Array Example

JAVASCRIPT SOURCE

JAVASCRIPT

```
const arr = [1, 2, 3];

const iterator = arr[Symbol.iterator]();

console.log(typeof arr[Symbol.iterator]);
console.log(typeof iterator.next);
```

Output:

▶ CONSOLE OUTPUT

OUTPUT

```
function
function
```

27.11 Custom Iterator

এখন আমরা নিজের Iterator বানাবো। ধরো:

► CONSOLE OUTPUT

OUTPUT

```
10
20
30
```

আমরা চাই object-টাকে `for ... of` দিয়ে iterate করতে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = {
  values: [10, 20, 30],
  [Symbol.iterator]() {
    let index = 0;
    return {
      next: () => {
        if (index < this.values.length) {
          return {
            value: this.values[index++],
            done: false
          };
        }
        return {
          value: undefined,
          done: true
        };
      }
    };
  }
};
```

এখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for (const number of numbers) {
  console.log(number);
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
10
20
30
```

🔗 এখানে কী হলো?

আমরা বললাম:

JAVASCRIPT SOURCE

JAVASCRIPT

```
[Symbol.iterator]()
```

যখন call হবে, তখন একটা iterator return করবে। Iterator-এর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
next()
```

method আছে। তারপর `for ... of` বারবার `next()` call করে।

27.12 Custom Iterator সহজ Version

একটু ছোট করে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const range = {  
  start: 1,  
  end: 5,  
  
  [Symbol.iterator]() {  
    let current = this.start;  
  
    return {  
      next: () => {  
        if (current ≤ this.end) {  
          return {  
            value: current++,  
            done: false  
          };  
        }  
        return {  
          done: true  
        };  
      }  
    };  
  }  
};  
  
for (const number of range) {  
  console.log(number);  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
1  
2  
3  
4  
5
```

27.13 Real-Life Example: Pagination

ধরো API থেকে data আসে:

► CONSOLE OUTPUT

OUTPUT

```
Page 1  
Page 2  
Page 3  
Page 4
```

Iterator concept দিয়ে আমরা page-by-page data process করতে পারি।

🕒 ARCHITECTURE / CONCEPT FLOW

```
Page 1 → process  
Page 2 → process  
Page 3 → process  
...
```

সব data একসাথে memory-তে রাখার প্রয়োজন নাও হতে পারে। এখান থেকেই আসে **lazy processing**!

27.14 Generator কী?

Generator হলো JavaScript-এর এমন একটি special function যেটা execution **pause এবং resume** করতে পারে। Generator function লিখতে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function*
```

ব্যবহার করা হয়। Example:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* numbers() {  
  
  yield 10;  
  yield 20;  
  yield 30;  
  
}
```

27.15 Generator Call

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* numbers() {  
  
  yield 10;  
  yield 20;  
  yield 30;  
  
}  
  
const generator = numbers();  
  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
{ value: 10, done: false }  
{ value: 20, done: false }  
{ value: 30, done: false }  
{ value: undefined, done: true }
```

27.16 yield কী?

yield generator-এর execution pause করে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* test() {  
  console.log("Step 1");  
  yield "Hello";  
  console.log("Step 2");  
  yield "World";  
  console.log("Step 3");  
}
```

এখন:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const generator = test();  
  
console.log(generator.next());  
console.log(generator.next());  
console.log(generator.next());
```

Output roughly:

► CONSOLE OUTPUT

OUTPUT

```
Step 1  
{ value: "Hello", done: false }  
  
Step 2  
{ value: "World", done: false }  
  
Step 3  
{ value: undefined, done: true }
```

🧠 Generator-এর Mental Model

🕒 ARCHITECTURE / CONCEPT FLOW

```
generator start  
↓  
yield 1  
↓  
pause  
↓  
next()  
↓  
yield 2  
↓  
pause  
↓  
next()  
↓  
yield 3  
↓  
pause  
↓  
next()  
↓  
done
```

27.17 Generator-এর সবচেয়ে গুরুত্বপূর্ণ বৈশিষ্ট্য

Normal function:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function test() {  
    return 10;  
    return 20;  
}
```

`return` করার পর function শেষ। কিন্তু Generator:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* test() {  
    yield 10;  
    yield 20;  
    yield 30;  
}
```

একাধিক value sequentially দিতে পারে।

27.18 Generator State

Generator state মনে রাখি।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* counter() {  
    let count = 1;  
    yield count++;  
    yield count++;  
    yield count++;  
}  
  
const generator = counter();  
  
console.log(generator.next().value);  
console.log(generator.next().value);  
console.log(generator.next().value);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
1  
2  
3
```

প্রতিবার state preserve হচ্ছে।

27.19 Generator + for ... of

Generator নিজেই iterable।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* numbers() {  
  yield 10;  
  yield 20;  
  yield 30;  
}  
  
for (const number of numbers()) {  
  console.log(number);  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
10  
20  
30
```

27.20 Generator + Array Spread

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* numbers() {  
  yield 10;  
  yield 20;  
  yield 30;  
}  
  
const result = [...numbers()];  
  
console.log(result);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
[10, 20, 30]
```

27.21 Generator Parameters

Generator-এর ভিতরে parameters নেওয়া যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* greeting(name) {  
  yield `Hello ${name}`;  
  yield "Welcome to JavaScript";  
}  
  
const generator = greeting("Shariar");  
  
console.log(generator.next().value);  
console.log(generator.next().value);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Hello Shariar  
Welcome to JavaScript
```

27.22 next(value)

এটি Generator-এর advanced feature। `next()` দিয়ে generator-এ value পাঠানো যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* test() {  
  const value = yield "Enter a value";  
  yield `You entered: ${value}`;  
}  
  
const generator = test();  
  
console.log(generator.next());  
console.log(generator.next(100));
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
{ value: "Enter a value", done: false }  
{ value: "You entered: 100", done: false }
```

♦ কীভাবে কাজ করছে?

প্রথম:

JAVASCRIPT SOURCE

JAVASCRIPT

```
generator.next()
```

Generator চলে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const value = yield "Enter a value";
```

এখানে pause। তারপর:

JAVASCRIPT SOURCE

JAVASCRIPT

```
generator.next(100)
```

এর 100 গিয়ে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
value
```

এর মধ্যে চুক যায়।

27.23 Generator return()

Generator manually stop করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* numbers() {  
  yield 1;  
  yield 2;  
  yield 3;  
}  
  
const generator = numbers();  
  
console.log(generator.next());  
  
console.log(generator.return("Stopped"));  
  
console.log(generator.next());
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
{ value: 1, done: false }  
{ value: "Stopped", done: true }  
{ value: undefined, done: true }
```

27.24 Generator throw()

Generator-এর ভিতরে error throw করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* test() {  
  try {  
    yield "Hello";  
  } catch (error) {  
    console.log("Error handled");  
  }  
}  
  
const generator = test();  
  
console.log(generator.next());  
  
generator.throw(new Error("Something went wrong"));
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
{ value: "Hello", done: false }  
Error handled
```

27.25 yield*

একটি generator অন্য generator-এর values yield করতে পারে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* numbers() {  
  yield 1;  
  yield 2;  
}  
  
function* allNumbers() {  
  yield* numbers();  
  yield 3;  
  yield 4;  
}  
  
for (const number of allNumbers()) {  
  console.log(number);  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
1  
2  
3  
4
```

27.26 Generator দিয়ে ID Generator

এটি খুব practical example।

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}

const generateId = idGenerator();

console.log(generateId.next().value);
console.log(generateId.next().value);
console.log(generateId.next().value);
console.log(generateId.next().value);
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
1
2
3
4
```

এখানে generator **infinite**। কিন্তু সমস্যা নেই, কারণ একসাথে সব generate করছে না। যখন দরকার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
next()
```

তখন value দিচ্ছে।

27.27 Infinite Generator

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* infiniteNumbers() {
  let number = 1;
  while (true) {
    yield number++;
  }
}
```

এটি theoretically infinite:

► CONSOLE OUTPUT

OUTPUT

```
1
2
3
4
5
...
∞
```

কিন্তু:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const generator = infiniteNumbers();

console.log(generator.next().value);
console.log(generator.next().value);
console.log(generator.next().value);
```

শুধু প্রয়োজনীয় তিনটি value তৈরি হয়েছে।

27.28 Lazy Evaluation

এটি Iterators/Generators-এর সবচেয়ে গুরুত্বপূর্ণ practical concept-এর একটি। **Lazy Evaluation** মানে: > প্রয়োজন হওয়ার আগে value তৈরি না করা। ধরো 1 থেকে 1 billion পর্যন্ত numbers। Normal array:

```
JAVASCRIPT SOURCE JAVASCRIPT

const numbers = [];

for (let i = 1; i ≤ 1_000_000_000; i++) {
  numbers.push(i);
}
```

এখানে huge memory লাগবে। Generator:

```
JAVASCRIPT SOURCE JAVASCRIPT

function* numbers() {

  let i = 1;

  while (true) {

    yield i++;

  }

}
```

এখানে সব number memory-তে একসাথে রাখা হচ্ছে না। যখন দরকার:

```
JAVASCRIPT SOURCE JAVASCRIPT

generator.next();
```

তখন value তৈরি হচ্ছে।

27.29 Practical Lazy Processing

ধরো আমরা শুধু প্রথম 5টি value চাই:

```
JAVASCRIPT SOURCE JAVASCRIPT

function* numbers() {

  let i = 1;

  while (true) {

    yield i++;

  }

}

const generator = numbers();

for (let i = 0; i < 5; i++) {

  console.log(generator.next().value);

}
```

Output:

```
► CONSOLE OUTPUT OUTPUT

1
2
3
4
5
```

27.30 Generator দিয়ে Fibonacci

Fibonacci:

► CONSOLE OUTPUT

OUTPUT

```
0
1
1
2
3
5
8
13
21
...
```

Generator:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* fibonacci() {

  let a = 0;
  let b = 1;

  while (true) {

    yield a;

    [a, b] = [b, a + b];

  }

}
```

ব্যবহার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const fib = fibonacci();

for (let i = 0; i < 10; i++) {

  console.log(fib.next().value);

}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
0
1
1
2
3
5
8
13
21
34
```

27.31 Generator দিয়ে Range

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* range(start, end, step = 1) {  
  for (let value = start; value ≤ end; value += step) {  
    yield value;  
  }  
}  
  
for (const number of range(1, 10, 2)) {  
  console.log(number);  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
1  
3  
5  
7  
9
```

এটি reusable utility হতে পারে।

27.32 Custom Iterable Class

Class-এর সাথে Iterator combine করা যায়।

JAVASCRIPT SOURCE

JAVASCRIPT

```
class NumberRange {  
  constructor(start, end) {  
    this.start = start;  
    this.end = end;  
  }  
  
  *[Symbol.iterator]() {  
    for (  
      let number = this.start;  
      number ≤ this.end;  
      number++  
    ) {  
      yield number;  
    }  
  }  
}  
  
const range = new NumberRange(1, 5);  
  
for (const number of range) {  
  console.log(number);  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
1  
2  
3  
4  
5
```

এখানে:

JAVASCRIPT SOURCE

JAVASCRIPT

```
*[Symbol.iterator]()
```

একটি generator method।

27.33 Generator Method

Class-এর method-ও generator হতে পারে।

JAVASCRIPT SOURCE

JAVASCRIPT

```
class UserList {  
  
  constructor(users) {  
    this.users = users;  
  }  
  
  *getUsers() {  
  
    for (const user of this.users) {  
  
      yield user;  
  
    }  
  
  }  
  
}  
  
const users = new UserList([  
  "Shariar",  
  "Rahim",  
  "Karim"  
]);  
  
for (const user of users.getUsers()) {  
  
  console.log(user);  
  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Shariar  
Rahim  
Karim
```

27.34 Iterator vs Generator

Iterator	Generator
next() manually implement করতে হয়	yield ব্যবহার করা যায়
বেশি code লাগে	কম code
Symbol.iterator manually লিখতে হতে পারে	generator automatically iterable
More control	Easier syntax
Low-level concept	Higher-level abstraction

♦ Iterator:

JAVASCRIPT SOURCEJAVASCRIPT

```
const iterator = {
  next() {
    return {
      value: 1,
      done: false
    };
  }
};
```

♦ Generator:

JAVASCRIPT SOURCEJAVASCRIPT

```
function* generator() {
  yield 1;
}
```

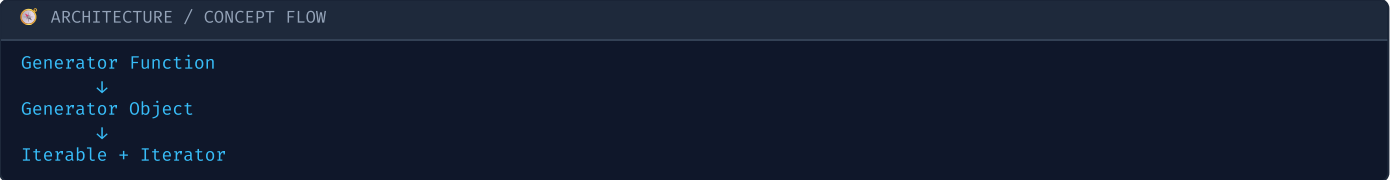
Generator অনেক ক্ষেত্রে iterator তৈরি করা সহজ করে।

27.35 Iterable vs Iterator vs Generator

এটা অবশ্যই clear রাখতে হবে।



Generator:



অর্থাৎ Generator object সাধারণত দুটোই: > **Iterable + Iterator**

27.36 কোনগুলো Iterable?

Value	Iterable?
Array	✓
String	✓
Map	✓
Set	✓
Generator	✓
Object {}	✗ সাধারণভাবে
Number	✗
Boolean	✗

এই জন্য:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for (const value of {}) {  
}
```

✗ TypeError হবে। কারণ সাধারণ object iterable নয়।

27.37 Object-কে Iterable করা

JAVASCRIPT SOURCE

JAVASCRIPT

```
const user = {  
  name: "Shariar",  
  age: 23,  
  *[Symbol.iterator]() {  
    yield this.name;  
    yield this.age;  
  }  
};  
  
for (const value of user) {  
  console.log(value);  
}
```

Output:

► CONSOLE OUTPUT

OUTPUT

```
Shariar  
23
```

এখন object iterable হয়ে গেছে।

27.38 Generator vs Normal Function

Normal Function	Generator
function	function*
return	yield
একবার execute	Pause/resume
সাধারণত single result	Multiple sequential results
state execution শেষে শেষ	state preserve করে
lazy sequence নয়	lazy sequence তৈরি করতে পারে

27.39 Real-Life Example: API Pagination

ধরো API-এর data:

▶ CONSOLE OUTPUT

OUTPUT

```
/page=1  
/page=2  
/page=3  
...
```

Conceptually:

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function* fetchPages() {  
  let page = 1;  
  while (true) {  
    const response =  
      await fetch(`/api/users?page=${page}`);  
    const data = await response.json();  
    if (data.length === 0) {  
      return;  
    }  
    yield data;  
    page++;  
  }  
}
```

ব্যবহার:

JAVASCRIPT SOURCE

JAVASCRIPT

```
for await (const users of fetchPages()) {  
  console.log(users);  
}
```

এটি **Async Generator**। এটা আমরা পরে Async JavaScript-এর সাথে আরও গভীরভাবে ব্যবহার করব।

27.40 Sync Generator বনাম Async Generator

♦ Sync Generator

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* numbers() {  
  yield 1;  
  yield 2;  
}
```

♦ Async Generator

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function* numbers() {  
  yield 1;  
  yield 2;  
}
```

Async generator asynchronous data stream-এর জন্য usefull যেমন:

▶ CONSOLE OUTPUT

OUTPUT

```
API pagination  
Database cursor  
Streaming data  
WebSocket-like data processing  
Large data processing
```

27.41 Generator কখন ব্যবহার করবে?

Generator useful হতে পারে যখন:

◆ 1. Large data

► CONSOLE OUTPUT

OUTPUT

Millions of records

◆ 2. Infinite sequence

► CONSOLE OUTPUT

OUTPUT

ID
Counter
Fibonacci

◆ 3. Lazy processing

► CONSOLE OUTPUT

OUTPUT

Generate only when needed

◆ 4. Pagination

► CONSOLE OUTPUT

OUTPUT

Page 1
Page 2
Page 3

◆ 5. Custom iteration

নিজের object-কে `for ... of` compatible করা।

27.42 Generator কখন ব্যবহার না করাই ভালো?

প্রতিটি সমস্যায় Generator দরকার নেই। যদি:

JAVASCRIPT SOURCE

JAVASCRIPT

```
const numbers = [1, 2, 3, 4, 5];
```

এতটুকুই হয়, তাহলে normal array যথেষ্ট। অথথা:

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* ...
```

ব্যবহার করলে code unnecessarily complex হতে পারে।

27.43 MUST KNOW

এই Chapter থেকে অবশ্যই মনে রাখবে:

- ♦ Iterable

JAVASCRIPT SOURCE

JAVASCRIPT

```
Symbol.iterator
```

থাকে।

- ♦ Iterator

JAVASCRIPT SOURCE

JAVASCRIPT

```
next()
```

থাকে।

- ♦ Iterator result

JAVASCRIPT SOURCE

JAVASCRIPT

```
{
  value,
  done
}
```

- ♦ Generator

JAVASCRIPT SOURCE

JAVASCRIPT

```
function* test() {}
```

- ♦ Yield

JAVASCRIPT SOURCE

JAVASCRIPT

```
yield value;
```

- ♦ Generator execution

JAVASCRIPT SOURCE

JAVASCRIPT

```
generator.next();
```

- ♦ Infinite Generator

JAVASCRIPT SOURCE

JAVASCRIPT

```
while (true) {
  yield value;
}
```

- ♦ Generator delegation

JAVASCRIPT SOURCE

JAVASCRIPT

```
yield* anotherGenerator();
```

- ♦ Async Generator

JAVASCRIPT SOURCE

JAVASCRIPT

```
async function* test() {}
```

Iteration Protocols এবং Generator স্টেট মেশিনের দ্রুত রেফারেন্সের জন্য সম্পূর্ণ ফ্লো ও সিনট্যাক্স সামারি:

🕒 ITERATION FLOW

```
Iterable
  ↓
Symbol.iterator
  ↓
Iterator
  ↓
next()
  ↓
{ value, done }
```

GENERATOR PATTERN

JAVASCRIPT

```
function* numbers() {
  yield 1;
  yield 2;
  yield 3;
}

const generator = numbers();
generator.next(); // { value: 1, done: false }
generator.next(); // { value: 2, done: false }
generator.next(); // { value: 3, done: false }
generator.next(); // { value: undefined, done: true }
```

yield

বর্তমান মান রিটার্ন করে এবং ফাংশনের এক্সিকিউশন স্টেট সাময়িকভাবে পজ (Pause) করে।

next()

পজ হওয়া জেনারেটর এক্সিকিউশনকে পুনরায় শুরু (Resume) করে পরবর্তী yield পর্যন্ত চালায়।

yield*

অন্য কোনো Iterable বা Generator-এ ডেলিগেট করে তার প্রতিটি মান ধারাবাহিকভাবে yield করে।

Iterators, Custom Iterables এবং Generators কনসেপ্টগুলোকে সম্পূর্ণ আয়ত্তে আনতে নিচে ৩টি লেভেলের ৯টি প্র্যাকটিক্যাল চ্যালেঞ্জের সম্পূর্ণ মডেল সলিউশন দেওয়া হলো:

Practice 1: Static Numeric Generator

BEGINNER

Problem Requirement: একটি জেনারেটর ফাংশন `generateTenToFifty()` তৈরি করো যা ক্রমান্বয়ে 10, 20, 30, 40, 50 মানগুলো `yield` দিয়ে প্রদান করবে।

PRACTICE 1 MODEL SOLUTION

✓ STANDARD PATTERN

```
function* generateTenToFifty() {
  yield 10;
  yield 20;
  yield 30;
  yield 40;
  yield 50;
}

const gen = generateTenToFifty();
for (const val of gen) {
  console.log(val);
}
```

▶ CONSOLE OUTPUT

OUTPUT

```
10
20
30
40
50
```

Practice 2: Loop-Driven Sequence Generator

BEGINNER

Problem Requirement: লুপের সাহায্যে একটি জেনারেটর ফাংশন `countToOneTen()` তৈরি করো যা 1 থেকে 10 পর্যন্ত সংখ্যা জেনারেট করবে।

PRACTICE 2 MODEL SOLUTION

✓ STANDARD PATTERN

```
function* countToOneTen() {
  for (let i = 1; i ≤ 10; i++) {
    yield i;
  }
}

const sequence = [...countToOneTen()];
console.log("Generated Sequence:", sequence);
```

▶ CONSOLE OUTPUT

OUTPUT

```
Generated Sequence: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Practice 3: Manual Iterator Traversal

BEGINNER

Problem Requirement: একটি সাধারণ অ্যারের `[Symbol.iterator]()` মেথড কল করে `while` লুপ ও `next()` দিয়ে ম্যানুয়ালি সব উপাদান প্রিন্ট করো।

PRACTICE 3 MODEL SOLUTION

✓ STANDARD PATTERN

```
const fruits = ["Apple", "Mango", "Banana"];

// Manually extract iterator from array
const iterator = fruits[Symbol.iterator]();

let result = iterator.next();
while (!result.done) {
  console.log(result.value);
  result = iterator.next();
}
```

▶ CONSOLE OUTPUT

OUTPUT

```
Apple
Mango
Banana
```

Practice 4: Custom range(start, end) Generator

INTERMEDIATE

Problem Requirement: Python-এর মতো একটি `range(start, end)` জেনারেটর ফাংশন বানাও যা `range(5, 10)` কল করলে 5 থেকে 10 রিটার্ন করবে।

PRACTICE 4 MODEL SOLUTION

✓ STANDARD PATTERN

```
function* range(start, end) {
  for (let current = start; current ≤ end; current++) {
    yield current;
  }
}

for (const num of range(5, 10)) {
  console.log(num);
}
```

▶ CONSOLE OUTPUT

OUTPUT

```
5
6
7
8
9
10
```

Practice 5: Zero-Padded User ID Generator

INTERMEDIATE

Problem Requirement: একটি ইনফিনিট আইডি জেনারেটর তৈরি করো যা কল করলে `USER-001`, `USER-002`, `USER-003` ফরম্যাটে প্যাডেড আইডি তৈরি করবে।

PRACTICE 5 MODEL SOLUTION

✓ STANDARD PATTERN

```
function* createUserIdGenerator(prefix = "USER") {
  let id = 1;
  while (true) {
    const formatted = String(id).padStart(3, "0");
    yield `${prefix}-${formatted}`;
    id++;
  }
}

const idGen = createUserIdGenerator();
console.log(idGen.next().value); // USER-001
console.log(idGen.next().value); // USER-002
console.log(idGen.next().value); // USER-003
console.log(idGen.next().value); // USER-004
```

▶ CONSOLE OUTPUT

OUTPUT

```
USER-001
USER-002
USER-003
USER-004
```

Practice 6: Fibonacci Infinite Generator

INTERMEDIATE

Problem Requirement: একটি ইনফিনিট ফিবোনাচ্চি জেনারেটর তৈরি করে তা থেকে প্রথম 10টি ফিবোনাচ্চি সংখ্যা সংগ্রহ করে প্রিন্ট করো।

PRACTICE 6 MODEL SOLUTION

✓ STANDARD PATTERN

```
function* fibonacci() {
  let current = 0;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const fib = fibonacci();
const firstTen = [];
for (let i = 0; i < 10; i++) {
  firstTen.push(fib.next().value);
}

console.log("First 10 Fibonacci:", firstTen);
```

▶ CONSOLE OUTPUT

OUTPUT

```
First 10 Fibonacci: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Practice 7: Dynamic Pagination Generator

ADVANCED

Problem Requirement: একটি `paginate(items, pageSize)` জেনারেটর ফাংশন তৈরি করো যা বড় অ্যারে থেকে পেজ অনুযায়ী চাঙ্ক (chunks) `yield` করবে।

PRACTICE 7 MODEL SOLUTION

✓ STANDARD PATTERN

```
function* paginate(items, pageSize = 3) {
  for (let i = 0; i < items.length; i += pageSize) {
    yield items.slice(i, i + pageSize);
  }
}

const dataset = [101, 102, 103, 104, 105, 106, 107];
const pager = paginate(dataset, 3);

console.log("Page 1:", pager.next().value);
console.log("Page 2:", pager.next().value);
console.log("Page 3:", pager.next().value);
console.log("End:", pager.next().done);
```

▶ CONSOLE OUTPUT

OUTPUT

```
Page 1: [101, 102, 103]
Page 2: [104, 105, 106]
Page 3: [107]
End: true
```

Practice 8: Custom Iterable Class Implementation

ADVANCED

Problem Requirement: একটি `NumberRange` ক্লাস তৈরি করো যা `[Symbol.iterator]()` মেথড ইমপ্লিমেন্ট করবে যাতে ক্লাসের ইন্সট্যান্স সরাসরি `for ... of` সমর্থন করে।

PRACTICE 8 MODEL SOLUTION

✓ STANDARD PATTERN

```
class NumberRange {
  constructor(from, to) {
    this.from = from;
    this.to = to;
  }

  *[Symbol.iterator]() {
    for (let current = this.from; current ≤ this.to; current++) {
      yield current;
    }
  }
}

const myRange = new NumberRange(1, 5);
const collected = [...myRange];
console.log("Spread from class:", collected);
```

▶ CONSOLE OUTPUT

OUTPUT

```
Spread from class: [1, 2, 3, 4, 5]
```

Practice 9: Async Generator for Streamed API Paging

ADVANCED

Problem Requirement: একটি `async function* fetchPages(totalPages)` জেনারেটর তৈরি করো যা অ্যাসিঙ্ক নেটওয়ার্ক রিকোয়েস্ট সিমুলেট করে `for await (const page of ...)` দিয়ে প্রসেস হবে।

PRACTICE 9 MODEL SOLUTION

✓ STANDARD PATTERN

```
// Simulating asynchronous API page delay
const wait = ms => new Promise(res => setTimeout(res, ms));

async function* fetchPages(totalPages = 3) {
  for (let p = 1; p <= totalPages; p++) {
    await wait(50); // Simulated network latency
    yield { page: p, data: [`Item ${p}A`, `Item ${p}B`] };
  }
}

async function runStream() {
  for await (const pageData of fetchPages(3)) {
    console.log(`Received Page ${pageData.page}:`, pageData.data);
  }
}

runStream();
```

► CONSOLE OUTPUT

OUTPUT

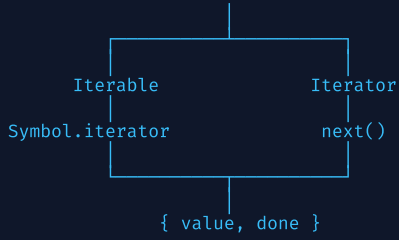
```
Received Page 1: ['Item 1A', 'Item 1B']
Received Page 2: ['Item 2A', 'Item 2B']
Received Page 3: ['Item 3A', 'Item 3B']
```

27.Map 🌀 Final Mental Map & Iteration Protocol Architecture

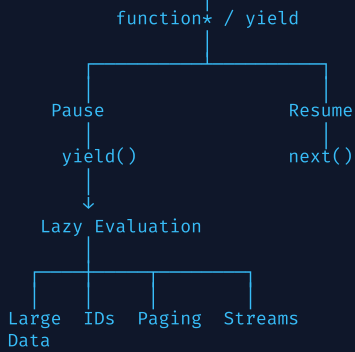
JavaScript Iteration Ecosystem-এর সম্পূর্ণ মেমরি ও কনসেপ্ট কার্ঠামোর চূড়ান্ত চিত্র:

🕒 ITERATORS & GENERATORS ECOSYSTEM MAP

Iteration



Generator



★ সবচেয়ে গুরুত্বপূর্ণ ৫টি Concept

যদি Chapter 27 থেকে শুধু ৫টা জিনিস মনে রাখতে চাও:

🌀 CORE CONCEPTS TAKEAWAY

QUICK RECALL

- 1 Iterable → Symbol.iterator
- 2 Iterator → next()
- 3 next() → { value, done }
- 4 Generator → function*
- 5 yield → value দেয় + execution pause করে

🌀 সবচেয়ে গুরুত্বপূর্ণ Relationship:

for...of → Iterable → Iterator → next()

এটা বুঝে ফেললে Chapter 27-এর মূল concept তোমার পরিষ্কার হয়ে যাবে।